不動産会社向け 応対トレーニングシステム

背景・目的

ある不動産会社では、営業担当者が入居者・オーナー・家主との応対を日常的に行っているが、個々の担当者の応対品質にばらつきがあり、組織として均質化・底上げが課題となっていた。従来の集合研修は時間・場所の制約が大きく、担当者一人ひとりの弱点に踏み込んだフィードバックを継続的に提供することが難しかった。

こうした課題を受け、Azure OpenAI（GPT）を活用した AI ロールプレイ研修システムを提案・開発した。担当者はブラウザ上で AI 顧客役との応対練習を繰り返し行い、会話終了後にミッション達成率・NG ワード使用有無・推奨フレーズの遵守状況などの評価結果をリアルタイムで受け取ることができる。シナリオはアバター動画・音声を用いた没入感のある形式で提供され、いつでも場所を問わず自己学習が可能な仕組みとなっている。

成果・課題

2025 年 9 月に本番リリースし、約 2,900 人の営業担当者を対象に全国事業所へ展開した。リリース後も継続的な運用と機能開発に従事している。研修成績の CSV 一括エクスポートや管理者向けダッシュボードも実装し、支店単位でのトレーニング状況を可視化できる体制を整えた。

技術面での主な課題は二点あった。一点目は LLM 評価の一貫性確保で、同一の応対に対する評価スコアが揺れないよう、プロンプト設計・ルーブリック定義・後処理ロジックを繰り返し改善した。二点目はシナリオ生成パイプラインのスケーラビリティで、TTS 音声生成→リップシンク動画生成という直列処理がリクエスト集中時に遅延するという問題に直面した。EventBridge を活用したイベント駆動型の非同期アーキテクチャへ再設計することで解決した（詳細は 2 枚目）。

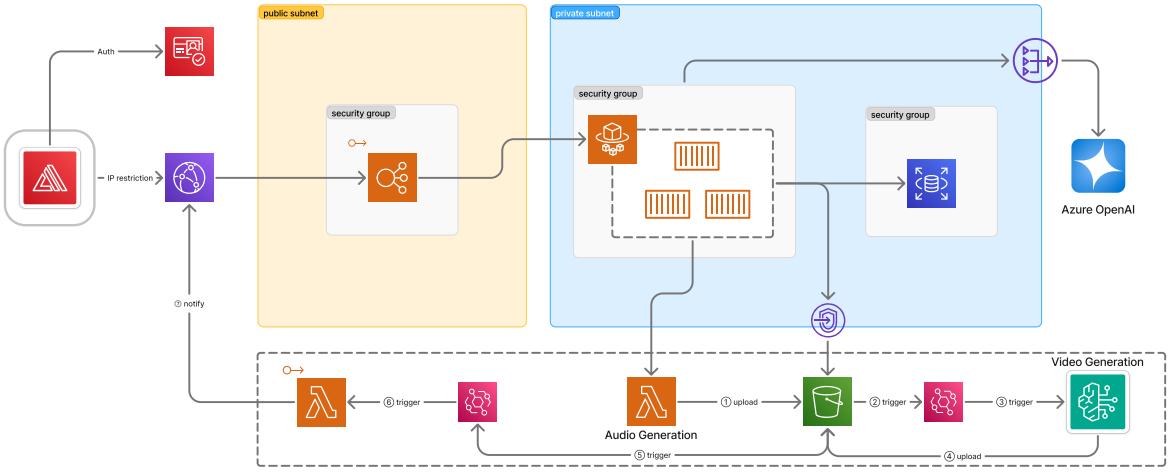
主な機能

- **AI ロールプレイ** — LLM が顧客役を演じ、テキストでインタラクティブな応対練習を提供
- **ミッション自動評価** — 会話終了時に GPT で各ミッションの達成度を自動採点（LLM-as-a-Judge）
- **NG ワード検出** — MeCab による形態素解析で禁止フレーズをリアルタイム検出
- **拒否フレーズ検出** — 顧客の断り発言を分類し、適切な切り返し応対をガイド
- **個別 AI フィードバック** — 会話終了後に GPT が応対の良否を具体的に解説
- **方言対応** — 地域ごとの方言でシナリオを提供し、より実践的な練習環境を実現
- **アバター動画生成** — TTS 音声合成 × リップシンクで没入感のあるシナリオ動画を自動生成
- **シナリオ管理** — 支店・事業部単位でロールプレイシナリオを管理・配信
- **成績レポート** — 研修結果の CSV 一括エクスポートと支店単位の管理者ダッシュボード
- **ユーザー管理** — Cognito による担当者・管理者のロールベース認証

プロジェクト概要

- **到達フェーズ：** 本番リリース済み（2025 年 9 月）・運用および機能開発継続中
- **開発体制：** チーム開発
- **担当ロール：** フルスタックエンジニア（フロントエンド・バックエンド・インフラ）、LLM エンジニア、ML エンジニア。Next.js フロントエンドの UI 設計・実装から FastAPI バックエンドの API 設計・実装、AWS インフラの Terraform 管理、LLM プロンプト設計と自動評価ロジックの実装まで、システム全体を横断して担当した。
- **開発期間：** 約 1 年間

アーキテクチャ概要



処理フロー

ロールプレイ：ユーザーの入力は CloudFront 経由で ECS 上の FastAPI に届き、Cognito + JWT による認証を経て Azure OpenAI API（GPT）に渡される。LLM が顧客役として応答を生成し、MeCab を用いた NG ワード検出とミッション達成判定を行ったのち、会話ログと評価結果を RDS（MySQL）に保存して返却する。

シナリオ生成（非同期パイプライン）：生成リクエストを受け付けると、セリフごとに Lambda（TTS）が音声を生成して S3 に保存する。S3 書き込みイベントを EventBridge がキャッチして SageMaker の非同期エンドポイントをトリガーし、リップシンク動画を生成する。動画が S3 に保存されると Lambda が再起動され、FastAPI を呼び出して完了を通知する。各セリフはジョブテーブルでステータス管理しており、パイプラインの各ステージで失敗ログと Slack 通知を設定することでサイレント障害を防いでいる。

クラウドサービス

- **AWS ECS/ECR** — FastAPI バックエンドを Docker コンテナで運用。CodeBuild + ECR で自動ビルド・デプロイ
- **AWS RDS (MySQL + PostgreSQL)** — 会話ログ・評価結果の永続化とセッションキャッシュ
- **AWS S3** — 音声・動画ファイルのオブジェクトストレージ。パイプラインのイベントソースとしても機能
- **AWS Lambda** — TTS 音声生成・完了通知のサーバーレス処理
- **AWS SageMaker（非同期エンドポイント）** — リップシンク動画生成。非同期で処理することでスケールを担保
- **AWS EventBridge** — S3 イベントを起点にパイプラインを連結するイベントルーティング
- **AWS Cognito** — ユーザー認証・JWT 発行・ロールベースのアクセス制御
- **AWS CloudFront** — CDN 配信。IP アドレス制限で社外からの API 直接呼び出しを遮断
- **Azure OpenAI (GPT)** — ロールプレイ会話生成・LLM-as-a-Judge による自動評価

セキュリティ・運用の工夫

インフラは全て Terraform で IaC 管理しており、パブリック/プライベートサブネットを分離した VPC 構成を採用している。外部からのアクセスは CloudFront の IP 制限とセキュリティグループで二重に制御し、社外ネットワークからの API 直接呼び出しを遮断している。認証は Cognito + JWT で行い、ロールに応じた細粒度のアクセス制御を実装した。運用面では、500 番台エラー発生時に Slack へ自動通知するミドルウェアを FastAPI に組み込み、シナリオ生成パイプラインの各ステージにも失敗ログと Slack アラートを設定することで、障害のサイレント化を防ぐ設計とした。